

When Training Data Are Gone: Selective Integration of Pretrained Intrusion Detectors

Abstract—Effective intrusion detection benefits from diverse attack data, yet raw historical source records may no longer be accessible due to privacy and retention constraints, preventing further collaborative training. In this paper, we study selective integration of authorized pretrained detectors when the raw source records are unavailable but pretrained detectors and coarse source class counts remain available for reuse. Under a hard deployment budget, the goal is to choose a fixed detector set that preserves diverse source attack-class coverage while maintaining predictive quality on current deployment data. We present SILOSTITCH, which gives weighted source class coverage strict priority over current predictive behavior rather than combining them into a single score. SILOSTITCH aligns heterogeneous detector outputs, uses held-out deployment records to assess current behavior, exactly maximizes weighted source class coverage under a logical byte budget via dynamic programming, and then refines the selected set without reducing coverage before fitting a shared stacker. To prevent from potential privacy leakage, we also protect uploaded score blocks via local differential privacy. Theoretically, we prove that calibration remains valid after adaptive selection and that the primary coverage problem is solved exactly, while perturbed scores satisfy label-conditional local differential privacy. Across four intrusion detection datasets with more than 60 candidate detectors per task, SILOSTITCH reduces communication by approximately half under a 50% selection-cost budget while preserving maximum weighted source coverage and near-full-pool detection quality.

I. INTRODUCTION

Effective intrusion detection relies on training data that capture diverse traffic conditions and attack behaviors. However, data collected by a single organization typically reflect only its own operational environment, limiting the diversity of attack patterns available for training. Collaborative learning, particularly Federated Learning (FL), can mitigate such data silos by learning from distributed records without pooling them centrally [1]–[4]. Yet these approaches still require contributors to retain local training records and participate in additional model optimization. In practice, raw historical network logs may become inaccessible because they contain sensitive information, such as internal network topologies and user identities [5], [6], and may need to be deleted under privacy and retention requirements such as the GDPR [7]. When these raw records are unavailable but authorized pretrained detectors and coarse source class counts remain, reusing the detectors offers a way to exploit complementary source knowledge without revisiting the original data.

Pretrained-model reuse can therefore operate after source records are no longer available [8], [9]. However, existing approaches either select or aggregate pretrained models without explicitly accounting for heterogeneous deployment costs [10],

[11], or perform cost-aware detector selection rather than choosing one fixed detector set for deployment [12]. This gap motivates our research question: *How can a coordinator selectively integrate a fixed set of pretrained intrusion detectors under a deployment budget while preserving source attack-class coverage and maintaining detection performance on the current deployment domain?*

Answering this question presents three challenges. First, detectors contributed by different parties may use different label spaces and support different attack classes, making their outputs difficult to compare and integrate directly. Second, source class coverage and current detection performance may favor different detectors: a detector with unique class coverage may no longer perform well after distribution shifts, whereas a detector that performs well on current deployment data may add little new coverage. The coordinator should therefore preserve unique source coverage without filling the limited budget with detectors that are currently weak or redundant. Third, detector selection must satisfy the deployment budget while remaining statistically reliable and privacy-aware. Because calibration data are used to choose detectors, their performance estimates must remain valid after selection, while uploaded prediction scores may still reveal sensitive information.

To address these challenges, we present SILOSTITCH, a framework for selective integration of frozen intrusion detectors under deployment budgets. The key idea is to prioritize source class coverage over current predictive performance: SILOSTITCH first maximizes weighted source class coverage under the deployment budget and then improves current predictive behavior without reducing that coverage. This priority separates historical class coverage from current predictive quality: coarse source class counts indicate which attack classes were represented during training, whereas held-out deployment records indicate how well the pretrained detectors perform after distribution shifts [13]. SILOSTITCH first aligns heterogeneous detector outputs to a common label space and uses held-out deployment records to assess their current predictive behavior. It then exactly solves the primary coverage problem through dynamic programming, refines the selected set without sacrificing coverage, and integrates the selected outputs with a shared stacker [14].

We further theoretically prove that calibration remains valid after adaptive detector selection and that the primary coverage problem is solved exactly. Because avoiding raw-data sharing alone does not eliminate privacy risks from model outputs [15], SILOSTITCH also supports optional Laplace perturbation before score upload. The perturbed scores satisfy

record-level, label-conditional local differential privacy; this guarantee applies to the uploaded scores, not to public labels or contributed detector parameters.

We evaluate SILOSTITCH on four intrusion detection datasets with over 60 candidate detectors per task. At a 50% selection cost budget, SILOSTITCH maximizes weighted source class coverage while reducing total logical communication by 49.3% to 50.4%, demonstrating that diverse source coverage can be retained at roughly half the communication cost. Importantly, these savings incur only a small loss in detection quality: across all four datasets, the macro F1 gap from the full detector pool is limited to 0.028. Consistent with the exactness guarantee, the primary solver attains the same coverage as exhaustive search in all comparison instances. In summary, we make four contributions.

- We formulate budget-constrained selective integration of pretrained intrusion detectors when raw historical source records are unavailable and introduce an ordered objective that prioritizes source class coverage over current predictive behavior.
- We develop SILOSTITCH to align heterogeneous detectors, exactly maximize weighted source coverage under the budget, refine the selected set without reducing coverage, and integrate their outputs with a shared stacker.
- We prove selection-valid calibration, exact primary coverage optimization, and local refinement stability, and establish label-conditional local differential privacy for randomized score uploads.
- We evaluate SILOSTITCH on four intrusion detection tasks, showing that a 50% selection-cost budget roughly halves communication while preserving maximum source coverage and near-full-pool detection quality.

II. BACKGROUND AND RELATED WORK

We first present and review existing collaborative training and model integration methods, and then introduce the notion of differential privacy.

Retraining and Adaptation of Intrusion Detector. When participants retain local training data and can rerun optimization, federated learning coordinates their parameter updates without transferring raw data [1]–[3]. Within intrusion detection, federated methods adapt this process to non-IID traffic and heterogeneous local learners [6], [19]. Recent NIDS research also redesigns the training pipeline to withstand adverse data conditions. For example, RAPIER handles low quality labels for encrypted malicious traffic [20], Rosetta uses TCP aware augmentation to improve robustness across network environments [21], and CoLD denoises labels before fitting a classifier [22]. These approaches improve how retained records are reused, but they still assume that participants can revisit the underlying training data after deployment. On the contrast, we consider the setting where the raw historical records are unavailable, while the pretrained detectors and coarse class-count metadata remain authorized for reuse.

Pretrained Model Integration. When source training data are no longer available but their trained models remain

accessible, pretrained model integration is the closest line of work. ZooD ranks a heterogeneous model zoo by cross domain transferability and aggregates its top K models [10], while other approaches combine pretrained models without accessing their source data [9], [11]. However, these methods primarily optimize transfer or integration utility and do not enforce a heterogeneous aggregate deployment cost budget over the selected model set and the score blocks required for integration.

Output Level Ensembles and Cost-Aware Selection. Once detector outputs are available, stacked generalization can train a downstream classifier over them [14]. SIRAJ aggregates reports from heterogeneous threat intelligence sources by pretraining an embedding and finetuning downstream predictors [16], while FedMStacking trains a global meta classifier from heterogeneous local predictions when labels are missing [18]. To account for inference cost, SPIREL learns a per item policy that trades off model invocations against classification utility [12]. These approaches cover complementary pieces of our setting, but not the same deployment decision. Output ensembles combine available predictions, and SPIREL controls cost through per-item invocation decisions. However, neither returns one fixed deployment subset after current-domain screening under a hard aggregate budget. SILOSTITCH instead makes a one-time subset decision and only then fits a shared classifier over the selected outputs.

Table I summarizes the distinctions most relevant to our deployment setting. Note that none of the compared methods jointly performs current deployment domain screening, fixed subset selection under a hard aggregate budget, strict source class priority, and shared stacking over only the selected outputs. This combination motivates the ordered selection problem formalized next.

Local Differential Privacy. Prediction outputs can reveal information about their inputs, so a privacy guarantee must specify both where randomization occurs and which fields it protects. For example, PrivateKT perturbs locally produced predictions before uploading them for knowledge transfer [17], whereas differentially private federated trees protect the statistics exchanged while training a new model [23]. Our setting differs because SILOSTITCH uploads scores from selected detectors to fit a shared classifier while treating labels as public. We therefore specialize the local model of Duchi et al. [24] to this release.

Definition II.1 (Label conditional local differential privacy). For a fixed public label y , a client applies a randomized mechanism $\mathcal{M}_y$ to the feature vector x . The upload $\mathcal{R}(x, y) = (\mathcal{M}_y(x), y)$ satisfies label conditional $(\epsilon, 0)$ -local differential privacy if, for every fixed y , every pair x, x' , and every measurable output set $\mathcal{O}$,

$$\Pr[\mathcal{M}_y(x) \in \mathcal{O}] \leq e^\epsilon \Pr[\mathcal{M}_y(x') \in \mathcal{O}]. \quad (1)$$

This definition compares records with the same public label and protects the feature derived portion of the upload, and the label itself remains public.

Table I
COMPARISON WITH RELATED METHODS. ✓ DENOTES DIRECT SUPPORT, ● RELATED SUPPORT, AND ○ NO SUPPORT.

Method	Model Integration			Subset Selection				Privacy	
	pretrained Models	Heterogeneous	Score Stacking	Current Screening	Fixed Set	Hard Budget	Class Priority	Score	LDP
SIRAJ [16]	○	✓	●	○	○	○	○	○	○
PrivateKT [17]	○	○	●	○	○	○	○	✓	○
SPIREL [12]	✓	✓	●	○	●	●	○	○	○
ZooD [10]	✓	✓	●	○	✓	●	○	○	○
FedMStacking [18]	○	✓	✓	○	○	○	○	○	○
SILOSTITCH	✓	✓	✓	✓	✓	✓	✓	✓	✓

To instantiate Definition II.1, let the deterministic score map g have replacement sensitivity

$$\Delta_1(g) = \sup_{x, x'} \|g(x) - g(x')\|_1. \quad (2)$$

Adding independent noise $\eta_k \sim \text{Lap}(0, \Delta_1(g)/\epsilon)$ to every coordinate yields a mechanism that satisfies Definition II.1 [25]. If one record releases blocks with parameters $\epsilon_1, \dots, \epsilon_s$, sequential composition gives the parameter $\sum_{i=1}^s \epsilon_i$. Under one record replacement, releases derived from unchanged records do not increase this privacy parameter. In SILOSTITCH, this mechanism applies to the selected detector score blocks. Theorem V.1 establishes label conditional local privacy for the resulting score upload under the public label adjacency relation.

III. PROBLEM STATEMENT

In this section, we formalize the problem of selective integration of pretrained intrusion detectors without historical training source. We first define the information available for detector reuse, map heterogeneous detector outputs to a common deployment label space, and model the communication cost of deploying a selected subset. We then specify the threat model for score uploads and formulate a budget-constrained selection objective that prioritizes source class coverage before current predictive behavior.

A. System Model

We consider J contributors (i.e., clients) that provide a pool of M pretrained intrusion detectors for integration. The raw records originally used to fit these detectors are no longer available to the integration process, while the metadata needed to interpret and deploy the detectors remain available. For candidate i , we denote its native probability output by $p_i(x) \in \mathbb{R}^{h_i}$, its serialized model size by K_i , and its coarse source class-count vector by $H_i = (H_{i,1}, \dots, H_{i,C})$ after semantic mapping. Because contributors may use different class definitions, we also associate each candidate with a validated semantic map that tells the coordinator how its native outputs correspond to the deployment label space.

We first make these heterogeneous outputs comparable. Let the deployment party contains C target classes. For candidate i , the coordinator validates a semantic map

$$A_i \in \{0, 1\}^{(C+1) \times h_i}, \quad \mathbf{1}^\top A_i = \mathbf{1}^\top, \quad (3)$$

and computes the aligned probability vector

$$q_i(x) = A_i p_i(x) \in \Delta^C, \quad (4)$$

where Δ^C is the probability simplex with $C + 1$ coordinates. The first C coordinates correspond to the predefined target classes. We reserve the final coordinate, denoted by $\perp$, for probability mass that cannot be mapped to a supported target class. By keeping this unsupported mass explicitly, later evaluation penalizes it instead of silently dropping it.

After making the detector outputs comparable, we use current deployment evidence to determine whether each pretrained detector remains useful under the current data distribution. Before selection, the coordinator therefore holds a labeled calibration set $\mathcal{D}^{\text{cal}}$ from the current deployment domain. We use these records only to evaluate the frozen candidate pool; they are disjoint from the records later used to fit the shared stacker and from the sealed test set for calibration. We also freeze every detector, semantic map, and source class count before calibration begins. The calibration labels remain hidden until both detector selection and stacker fitting are complete, so they cannot affect any selection decision.

We next model the deployment cost of selecting a detector. A selected detector must be delivered for use, and its aligned probability block must later be uploaded for shared stacker fitting. We combine these two communication terms into a single logical cost. Let B_t denote the total number of stacker-fitting records across all contributors, and let d be the logical byte width of one probability value. For a selected set S , we define

$$C_{\text{sel}}(S) = \sum_{i \in S} b_i, \quad b_i = JK_i + dB_t(C + 1). \quad (5)$$

The first term in b_i accounts for delivering candidate i to the J clients, while the second accounts for uploading one aligned score block for every stacker-fitting record.

Given a budget fraction β , we set the hard selection budget to

$$B = \left\lceil \beta \sum_{i=1}^M b_i \right\rceil. \quad (6)$$

The reference cost includes the entire candidate pool, including candidates that may later fail calibration. We additionally

count the initial candidate upload and the stacker-label upload:

$$C_{\text{total}}(S) = \sum_{i=1}^M K_i + C_{\text{sel}}(S) + dB_t. \quad (7)$$

We use C_{sel} to enforce the deployment budget and C_{total} to compare the overall logical communication of SILOSTITCH with that of the full detector pool. For later selection, we will convert each coarse class count $H_{i,c}$ into a binary source-support indicator $\Gamma_{i,c}$ and derive a class weight w_c from the aggregate source counts. We introduce these two quantities here so that the workflow notation is easy to follow; their exact definitions appear in the coverage-selection subsection.

B. Threat Model

We consider the privacy risk introduced when contributors upload detector scores for shared stacker fitting. Although raw historical source records are unavailable and never transferred during integration, the prediction scores computed from current deployment records may still reveal information about those records. We therefore consider a coordinator that follows the prescribed integration protocol but may attempt to infer record-level information from the uploaded scores.

We assume that each contributed detector, its semantic map, and its coarse source class counts are fixed and honestly reported before selection. Record labels used for stacker fitting are treated as public. When score protection is enabled, each contributor randomizes its selected score blocks locally before transmission, so the privacy guarantee does not rely on the coordinator performing the randomization. Our guarantee protects the feature-dependent information contained in these randomized score uploads; it does not protect the public labels, contributed detector parameters, or source class-count metadata.

We do not consider malicious contributors that poison detector parameters, forge semantic maps or source class counts, or deviate from the prescribed randomization mechanism. We leave these integrity attacks outside our scope because we focus on record-level privacy of score uploads.

C. Detector Selection Objective

The aligned outputs let us compare detectors in a common label space, but the budget may still prevent us from deploying all of them. We therefore need to decide which subset to keep. Two considerations matter: we want the selected set to preserve attack classes represented across the source domains, and we also want its detectors to remain useful on current deployment data. These goals can favor different candidates. For example, one detector may be the only affordable model that covers a rare source class but have only moderate current performance, while another may perform well now but add no new source-class coverage.

We handle this conflict with a strict priority instead of a direct tradeoff. We first maximize weighted source class coverage under the deployment budget. Only after that maximum coverage value has been fixed do we compare current predictive behavior. This is a lexicographic objective: the

secondary objective can improve the choice among equally well-covered sets, but it can never compensate for losing primary coverage. Formally, we target

$$\text{lexmax}_{\emptyset \neq S \subseteq \widehat{\mathcal{R}}} (C_{\text{sem}}(S), \Phi(S)) \quad \text{s.t.} \quad \sum_{i \in S} b_i \leq B. \quad (8)$$

Here, $\widehat{\mathcal{R}}$ is the pool that passes current-data calibration, C_{sem} is the weighted source class coverage defined later, and Φ summarizes current predictive behavior. We solve the maximum source-coverage problem exactly and then improve current predictive behavior without reducing the achieved coverage. Section V later explains why a single fixed weighted sum cannot enforce this strict ordering for every possible candidate pool.

IV. DESIGN OF SILOSTITCH

A. Overview of SILOSTITCH

We now describe how SILOSTITCH realizes the preceding problem formulation as an end-to-end integration workflow. Figure 1 summarizes the workflow in four steps. In Step 1, we use the semantic map A_i defined in the system model to transform each detector's native probabilities $p_i(x)$ into the aligned representation $q_i(x)$, so that heterogeneous detector outputs can be compared in a common label space. We also retain the source class-count information and logical cost introduced earlier for subsequent selection. From these source counts, we later derive the source-support masks and class weights used by the coverage objective.

With the detector outputs made comparable, Step 2 uses held-out calibration records to evaluate how the frozen detectors behave on current deployment data. We use a Brier-risk upper confidence bound to determine whether each detector enters the calibration-eligible pool, while retaining additional classwise and behavioral statistics for later refinement. Based on this eligible pool, Step 3 first uses the source-support information and detector costs to find the maximum weighted source class coverage allowed by the deployment budget. We then use the retained calibration statistics to improve current predictive behavior without reducing the achieved coverage. Finally, once the selected detector set is fixed, Step 4 uses only its aligned probability blocks to fit one shared stacker and keeps the same selected layout for subsequent sealed-test inference. The following subsections define the operations in these four steps in detail.

B. Selection and Stacking Workflow

The selection and stacking stages use different records because they answer different questions. We use the calibration partition to decide which frozen detectors should be eligible and which subset should be selected. Only after we freeze that subset do we use the independent stacker-fitting partition to train the final combiner. This separation prevents the data used to fit the stacker from also influencing which detector outputs become its input features.

Algorithm 2 follows the same order as Figure 1. We first calibrate the candidate pool, then compute the exact primary

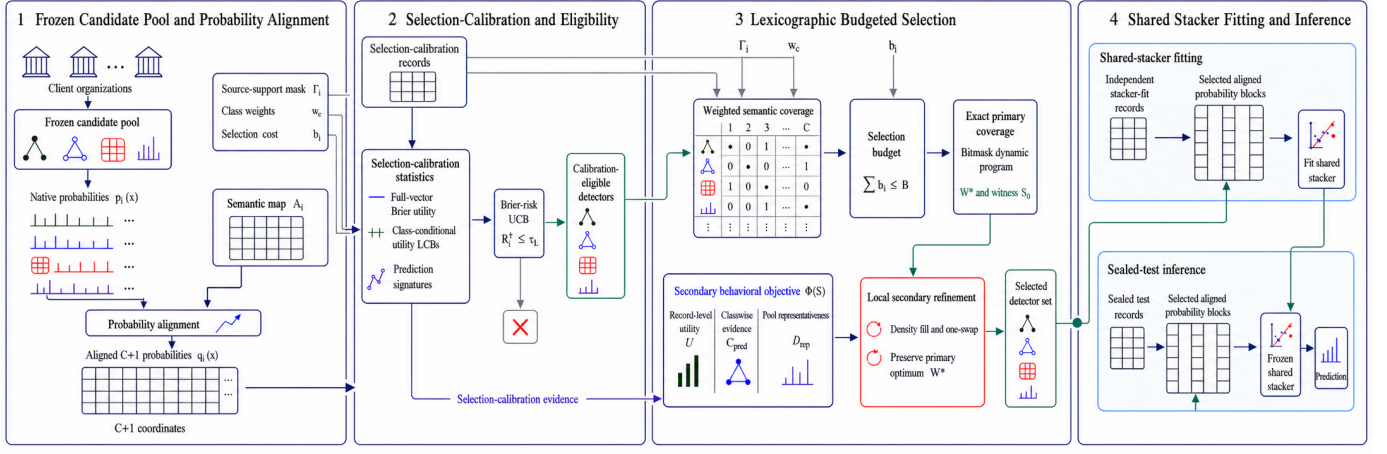


Figure 1. Overview of SILOSTITCH. Step 1 maps the heterogeneous outputs of the frozen detector pool to a common $C + 1$ -coordinate label space and retains the metadata needed for selection. Step 2 evaluates the aligned detectors on held-out calibration records and forms the calibration-eligible pool. Step 3 exactly maximizes weighted source class coverage under the deployment budget and then refines current predictive behavior without reducing the achieved coverage. Step 4 fits a shared stacker from the selected aligned outputs and reuses the same selected layout for sealed-test inference.

Algorithm 1 SILOSTITCH Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^M, \mathcal{D}^{\text{cal}}, \{\mathcal{D}_j^{\text{stk}}\}_{j=1}^J, B, \epsilon_b \in (0, \infty]$
Ensure: $\hat{S}, \hat{f}$
1: Clients $\rightarrow$ Server: $\{f_i, A_i, H_i, K_i\}_{i=1}^M$
2: $(\hat{\mathcal{R}}, \Gamma, w, b, \Phi) \leftarrow \text{CALIBRATE}(\mathcal{D}^{\text{cal}})$
3: $(W^*, S_0) \leftarrow \text{COVER}(\hat{\mathcal{R}}, \Gamma, w, b, B)$
4: $\hat{S} \leftarrow \text{REFINE}(S_0, W^*, \Phi, B)$
5: Server $\rightarrow$ Clients: $\{f_i, A_i\}_{i \in \hat{S}}$
6: **for** $j = 1, \dots, J$ **in parallel do**
7: $Z_j \leftarrow \text{concat}_{i \in \hat{S}} q_i(X_j^{\text{stk}})$
8: $(\Xi_j)_{t,i,k} \stackrel{\text{iid}}{\sim} \text{Lap}(0, 2/\epsilon_b)$
9: $\tilde{Z}_j \leftarrow Z_j + \Xi_j$
10: Client $j \rightarrow$ Server: $(\tilde{Z}_j, Y_j^{\text{stk}})$
11: **end for**
12: $\hat{f} \leftarrow \text{FIT}(\bigcup_j (\tilde{Z}_j, Y_j^{\text{stk}}))$
13: **return** $(\hat{S}, \hat{f})$

Algorithm 2 SILOSTITCH Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^M, \mathcal{D}^{\text{cal}}, \{\mathcal{D}_j^{\text{stk}}\}_{j=1}^J, B, \epsilon_b \in (0, \infty]$
Ensure: $\hat{S}, \hat{f}$
1: Clients $\rightarrow$ Server: $\{f_i, A_i, H_i, K_i\}_{i=1}^M$
2: $(\hat{\mathcal{R}}, \Gamma, w, b, \Phi) \leftarrow \text{CALIBRATE}(\mathcal{D}^{\text{cal}})$
3: $(W^*, S_0) \leftarrow \text{COVER}(\hat{\mathcal{R}}, \Gamma, w, b, B)$
4: $\hat{S} \leftarrow \text{REFINE}(S_0, W^*, \Phi, B)$
5: Server $\rightarrow$ Clients: $\{f_i, A_i\}_{i \in \hat{S}}$
6: **for** $j = 1, \dots, J$ **in parallel do**
7: $Z_j \leftarrow \text{concat}_{i \in \hat{S}} q_i(X_j^{\text{stk}})$
8: $(\Xi_j)_{t,i,k} \stackrel{\text{iid}}{\sim} \text{Lap}(0, 2/\epsilon_b)$
9: $\tilde{Z}_j \leftarrow Z_j + \Xi_j$
10: Client $j \rightarrow$ Server: $(\tilde{Z}_j, Y_j^{\text{stk}})$
11: **end for**
12: $\hat{f} \leftarrow \text{FIT}(\bigcup_j (\tilde{Z}_j, Y_j^{\text{stk}}))$
13: **return** $(\hat{S}, \hat{f})$

coverage value, and then refine one exact primary solution using current predictive evidence. After the selected set $\hat{S}$ is fixed, the clients evaluate only those detectors on their stacker-fitting records. If privacy protection is enabled, they randomize the selected aligned score blocks locally before upload; otherwise the same workflow is used without added noise.

C. Probability Alignment and Brier Calibration

We have already aligned every candidate to the same target label space through $q_i(x)$. We now need current-data scores that can be compared across these heterogeneous detectors. We use a full-vector Brier score because it is bounded, evaluates the complete aligned probability vector, and naturally penalizes probability mass assigned to the unsupported $\perp$ coordinate.

For calibration record (x_t, y_t) , let $e_{y_t} \in \mathbb{R}^{C+1}$ be the one-hot label vector with a zero $\perp$ coordinate. For candidate i , we

define

$$\ell_{i,t} = \frac{1}{2} \|q_i(x_t) - e_{y_t}\|_2^2, \quad g_{i,t} = 1 - \ell_{i,t} \in [0, 1]. \quad (9)$$

Thus, $g_{i,t}$ measures the quality of the whole aligned prediction rather than one coordinate at a time. If a detector places probability on an unsupported output, that probability remains in the vector and lowers the utility.

Eligibility screening. We first decide whether each frozen detector is reliable enough to remain in the candidate pool. Using only its empirical mean can be misleading when the calibration set is finite, so we screen with an upper confidence bound on current Brier risk. With N calibration records and failure probability $\delta \in (0, 1)$, we compute

$$\hat{\mu}_i = \frac{1}{N} \sum_{t=1}^N g_{i,t}, \quad r = \sqrt{\frac{\log(4M/\delta)}{2N}}, \quad (10)$$

and define

$$R_i^+ = 1 - \hat{\mu}_i + r. \quad (11)$$

We keep candidate i only if $R_i^+ \leq \tau_L$, giving the calibration-eligible pool

$$\widehat{\mathcal{R}} = \{i : R_i^+ \leq \tau_L\}. \quad (12)$$

In other words, eligibility depends on both the observed current performance and the uncertainty caused by finite calibration data.

Classwise scores. An acceptable average risk does not guarantee that a detector remains useful for every attack class. We therefore keep a conservative current score for each detector–class pair and use these scores later when we refine sets that already achieve maximum source coverage. For class c , let

$$N_c = \sum_t \mathbf{1}\{y_t = c\}, \quad (13)$$

and define

$$\widehat{\mu}_{i,c} = \frac{1}{N_c} \sum_{t:y_t=c} g_{i,t}. \quad (14)$$

We require every calibration class to appear at least once, so $N_c > 0$. We then compute

$$r_c = \sqrt{\frac{\log(4MC/\delta)}{2N_c}}, \quad a_{i,c} = [\widehat{\mu}_{i,c} - r_c]_+, \quad (15)$$

where $[z]_+ = \max\{0, z\}$. The value $a_{i,c}$ is a lower-confidence estimate of the current utility of candidate i on class c .

Behavioral similarity. Two detectors can have similar average utility and still make nearly identical predictions. Selecting both may then consume budget without adding much new behavior. To capture this redundancy, we compare the aligned outputs of the candidates on a deterministic class-stratified anchor set $\mathcal{A}$. We form

$$z_i = \text{vec}([q_i(x_t)]_{t \in \mathcal{A}}), \quad \kappa_{i,k} = \left[\frac{\langle z_i, z_k \rangle}{\|z_i\|_2 \|z_k\|_2} \right]_0^1. \quad (16)$$

The clipping only removes numerical roundoff outside $[0, 1]$. Later, we use these similarities in a representativeness term: rather than rewarding pairwise difference for its own sake, we ask whether the selected set contains detectors whose behavior represents the wider eligible pool.

D. Coverage-Prioritized Detector Selection

The calibration statistics tell us how the frozen detectors behave on current deployment data, but they do not tell us which historical attack classes would disappear if particular detectors were removed. That information comes from the coarse source class counts retained with the candidate models. We therefore use source class coverage as the primary objective and use the calibration statistics only after the best achievable coverage has been fixed.

Weighted source class coverage. For candidate i and target class c , let $H_{i,c}$ be its accepted source-side count after semantic mapping. A very small count may provide weak evidence that a class was meaningfully represented during training, so we first convert the count into a support indicator. For a minimum support threshold $n_{\min} > 0$, define

$$\Gamma_{i,c} = \mathbf{1}\{H_{i,c} \geq n_{\min}\}. \quad (17)$$

Thus, $\Gamma_{i,c} = 1$ means that candidate i carries enough historical support for class c to count toward the primary objective.

We also give more weight to source classes that are scarce across the whole candidate pool. Let $H_c = \sum_i H_{i,c}$ denote the aggregate source count of class c . For $H_c > 0$, we set

$$w_c = \frac{H_c^{-1/2}}{\sum_{d=1}^C H_d^{-1/2}}, \quad \sum_{c=1}^C w_c = 1. \quad (18)$$

The inverse-square-root rule increases the importance of less represented classes without letting one extremely rare class dominate the whole objective solely because its raw count is small.

For selected set S , we define weighted source class coverage as

$$C_{\text{sem}}(S) = \sum_{c=1}^C w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1\}. \quad (19)$$

A class contributes once as soon as at least one selected detector provides sufficient source support for it; selecting more detectors for the same class does not increase the primary value. We exclude the unsupported $\perp$ coordinate. Therefore, C_{sem} measures how much historical attack-class coverage is retained by the selected set, not how accurately that set predicts current records.

Secondary behavioral objective. After the primary coverage value is fixed, several feasible detector sets may still attain exactly the same coverage. We compare such sets using three current-data signals that capture different failure modes: overall utility, weak-class behavior, and redundant behavior. With $h_\tau(z) = 1 - e^{-z/\tau}$, define

$$\begin{aligned} U(S) &= \frac{1}{N} \sum_{t=1}^N h_{\tau_U} \left(\sum_{i \in S} g_{i,t} \right), \\ C_{\text{pred}}(S) &= \frac{1}{C} \sum_{c=1}^C h_{\tau_C} \left(\sum_{i \in S} a_{i,c} \right), \\ D_{\text{rep}}(S) &= \frac{1}{|\widehat{\mathcal{R}}|} \sum_{k \in \widehat{\mathcal{R}}} \max_{i \in S} \kappa_{i,k}. \end{aligned} \quad (20)$$

The term $U(S)$ rewards sets whose members perform well across the calibration records. The term $C_{\text{pred}}(S)$ keeps attention on classwise behavior by using the conservative scores $a_{i,c}$. The term $D_{\text{rep}}(S)$ rewards sets whose prediction patterns represent the behavior found across the eligible pool.

The saturating functions in U and C_{pred} impose diminishing returns. Once several selected detectors already perform well on the same record or class, another similar detector contributes less. This makes it harder for repeated versions of the same behavior to dominate the secondary comparison. We define $\max_{i \in \emptyset} \kappa_{i,k} = 0$ and combine the three terms as

$$\Phi(S) = \lambda_U U(S) + \lambda_C C_{\text{pred}}(S) + \lambda_D D_{\text{rep}}(S), \quad (21)$$

where the nonnegative weights sum to one. Crucially, we use Φ only after the primary coverage value has been fixed, so no gain in current predictive behavior can compensate for a loss in source class coverage.

Budgeted selection procedure. We now solve the primary problem: among the calibration-eligible candidates, what is the largest weighted source class coverage that fits within the deployment budget? Define

$$\mathcal{F}_B = \left\{ \emptyset \neq S \subseteq \hat{\mathcal{R}} : \sum_{i \in S} b_i \leq B \right\}, \quad (22)$$

$$W^* = \max_{S \in \mathcal{F}_B} C_{\text{sem}}(S).$$

Because C_{sem} depends only on which classes are covered, we encode every support vector Γ_i as a C -bit mask and run a sparse dynamic program over reachable masks. For each mask, we retain only the least-cost detector set that produces it. Any more expensive set with the same mask is dominated because every future detector changes the two masks in exactly the same way.

The dynamic program returns both the exact optimum W^* and a least-cost set S_0 that attains it. Starting from this least-cost exact solution leaves as much unused budget as possible for improving current predictive behavior. We first consider additional candidates according to their secondary gain per byte and accept an addition only when its absolute gain exceeds ϵ_{loc} . We then examine one-for-one replacements and accept a swap only when it remains budget feasible, keeps coverage equal to W^* , and improves Φ by more than the same tolerance. Every accepted refinement therefore improves current behavior without weakening the exact primary guarantee.

For the support-priority ablation in the evaluation, we apply the same secondary refinement without enforcing the primary source-coverage requirement. We use this variant only to isolate the effect of strict coverage priority; it is not an additional branch of the deployed SILOSTITCH workflow.

E. Shared Stacker Learning

Once the selector freezes $\hat{S}$, only the aligned outputs of the selected detectors enter the shared learner. For input x , we define the selected representation

$$\phi_{\hat{S}}(x) = \text{concat}_{i \in \hat{S}} q_i(x) \in \mathbb{R}^{|\hat{S}|(C+1)}. \quad (23)$$

The selected detector set therefore fixes both which probability blocks are used and the order in which they appear before the stacker is fitted.

Local score perturbation. Even though the raw source records are not shared, a prediction score can still reveal information about the record that produced it. When score protection is enabled, each client therefore randomizes every coordinate of each selected score block before upload:

$$\tilde{q}_i(x_t) = q_i(x_t) + \eta_{i,t}, \quad \eta_{i,t,k} \stackrel{\text{iid}}{\sim} \text{Lap}\left(0, \frac{2}{\epsilon_b}\right). \quad (24)$$

The resulting stacker input is

$$\tilde{\phi}_{\hat{S}}(x) = \text{concat}_{i \in \hat{S}} \tilde{q}_i(x). \quad (25)$$

Section V later proves the corresponding label-conditional local privacy guarantee and its composition across selected blocks.

Shared stacker fitting. After selection is complete, the independent stacker-fitting partition $\mathcal{D}^{\text{stk}}$ supplies the labels and selected score features used to train one shared classifier:

$$\hat{f}_{\hat{S}} = \text{Fit}_{\text{global}}\left(\{(\tilde{\phi}_{\hat{S}}(x_t), y_t) : (x_t, y_t) \in \mathcal{D}^{\text{stk}}\}\right). \quad (26)$$

For the nonprivate configuration, we simply set $\tilde{\phi}_{\hat{S}} = \phi_{\hat{S}}$. Because both configurations use only the columns indexed by $\hat{S}$, unselected detectors cannot influence the learned stacker.

During deployment, clients would generate the Laplace noise locally before transmission. Our centralized privacy experiment samples the same distribution only to isolate the utility effect of score randomization. For sealed-test inference, we keep the selected detector set and aligned feature layout fixed and reuse the fitted shared stacker without retraining any contributed detector.

V. THEORETICAL ANALYSIS

We now analyze the guarantees that support the workflow above. We follow the same order as the design so that each result answers a concrete question raised by the implementation. First, we show what the local score randomization protects. Second, we show that the calibration bounds remain valid even after we use those same calibration statistics to choose detectors. Third, we prove that the primary coverage solver is exact and characterize what the local secondary refinement guarantees. Finally, we examine how a change in the client mixture can affect the interpretation of the selected detectors after deployment.

A. Local Privacy of Score Uploads

Recall that the shared stacker is trained from selected detector score blocks rather than from the historical source records. Because these score blocks are still record dependent, we ask what information the client-side Laplace mechanism protects. We use the label-conditional adjacency relation defined earlier: the record label is public, and two neighboring records (x, y) and (x', y) differ only in their feature vectors.

Before any stacker-fitting record is processed, we fix the selected detector set and all semantic maps. For a batched upload, neighboring batches have the same size, the same row alignment, and the same sequence of public labels, but they differ in one feature vector. Each client samples fresh noise independently across records, selected detector blocks, and output coordinates. Under this execution model, we obtain the following guarantee.

Theorem V.1 (Label-conditional local score privacy). Suppose that the selected detectors and their semantic maps are fixed and that $0 < \epsilon_b < \infty$, and suppose that each client uses fresh private randomness in Eq. 24. Then one released detector block satisfies $(\epsilon_b, 0)$ -local differential privacy under the preceding label-conditional adjacency relation. If $s = |\hat{S}|$ blocks are released for the same record, their joint release satisfies $(s\epsilon_b, 0)$ -local differential privacy. Under one-record replacement, releasing the entire client batch has the same

record-level guarantee. More generally, assigning parameter ϵ_i to block i gives $(\sum_{i \in \hat{S}} \epsilon_i, 0)$ -local differential privacy.

Proof. Because $q_i(x)$ and $q_i(x')$ are probability vectors, their replacement sensitivity satisfies

$$\|q_i(x) - q_i(x')\|_1 \leq 2. \quad (27)$$

Therefore, adding coordinatewise Laplace noise with scale $2/\epsilon_b$ bounds the density ratio of one released block by e^{ϵ_b} . Sequential composition over the s selected blocks gives the factor $e^{8\epsilon_b}$.

For a batched upload, neighboring batches differ in only one row. Since different rows use independent randomization, parallel composition across the unchanged rows does not increase the record-level privacy parameter. Finally, appending the unchanged public labels does not change the density ratio. $\square$

The theorem also clarifies how the privacy budget interacts with detector selection. If we want one total per-record score budget ϵ , we can assign $\epsilon_b = \epsilon/s$ to each of the s selected blocks. The corresponding noise scale becomes $2s/\epsilon$. Thus, under a fixed total score-privacy budget, selecting fewer detector blocks also reduces the noise that each released block requires. The guarantee applies only to the randomized score upload, and privacy loss composes again if the same record is released in repeated rounds.

B. Calibration under Heterogeneous Clients

The privacy result concerns the stacker-fitting partition. Detector selection, in contrast, depends on the separate calibration partition introduced in the system model. We therefore ask a different question here: after we use the calibration statistics to choose a detector subset, can we still trust the confidence bounds computed before selection?

The calibration records may come from different clients and classes, so we do not assume that every record follows one identical distribution. Instead, we condition on the observed client-label design

$$\mathcal{I} = \{(j_t, y_t)\}_{t=1}^N \quad (28)$$

and on the frozen candidate pool $\mathcal{Q}$. Conditioned on $(\mathcal{Q}, \mathcal{I})$, we assume the records are independent, while allowing their conditional distributions to differ across records. We define

$$\mu_i = \frac{1}{N} \sum_{t=1}^N \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}], \quad R_i = 1 - \mu_i, \quad (29)$$

and

$$\mu_{i,c} = \frac{1}{N_c} \sum_{t: y_t=c} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}]. \quad (30)$$

These quantities describe the record-weighted calibration mixture actually observed by the coordinator. They should not be read as equal-client averages or worst-client performance.

Theorem V.2 (Simultaneous calibration). Under the preceding assumptions, with probability at least $1 - \delta$, the following

inequalities hold simultaneously for every candidate i and class c :

$$R_i \leq R_i^+ \leq R_i + 2r, \quad (31)$$

$$\max\{0, \mu_{i,c} - 2r_c\} \leq a_{i,c} \leq \mu_{i,c}. \quad (32)$$

Consequently, these inequalities remain valid for every candidate in any set selected adaptively from the calibration statistics.

Proof. Conditioned on $(\mathcal{Q}, \mathcal{I})$, Hoeffding's inequality gives

$$\Pr\{|\hat{\mu}_i - \mu_i| > r \mid \mathcal{Q}, \mathcal{I}\} \leq 2e^{-2Nr^2} = \frac{\delta}{2M}.$$

A union bound over the M candidates therefore uses at most $\delta/2$ failure probability. Likewise,

$$\Pr\{|\hat{\mu}_{i,c} - \mu_{i,c}| > r_c \mid \mathcal{Q}, \mathcal{I}\} \leq 2e^{-2N_c r_c^2} = \frac{\delta}{2MC},$$

so a second union bound over all MC candidate-class pairs uses the remaining $\delta/2$. Thus, with probability at least $1 - \delta$, all overall and classwise inequalities hold at the same time.

On this event, $1 - \hat{\mu}_i + r$ lies between R_i and $R_i + 2r$, while $\hat{\mu}_{i,c} - r_c$ lies between $\mu_{i,c} - 2r_c$ and $\mu_{i,c}$. Applying $[\cdot]_+$ gives the classwise result. The key point is that this one event covers all $M + MC$ quantities before selection occurs. We can therefore choose a data-dependent subset without paying another union bound over the 2^M possible detector sets. $\square$

The same simultaneous event also tells us how the eligibility screen can affect the best source coverage available to the selector. For any candidate pool $\mathcal{E}$, let $W(\mathcal{E})$ denote the largest budget-feasible weighted source class coverage available from that pool, with value zero if no nonempty set is feasible. Define

$$\mathcal{R}^- = \{i : R_i \leq \tau_L - 2r\}, \quad \mathcal{R}^0 = \{i : R_i \leq \tau_L\}. \quad (33)$$

Then

$$\mathcal{R}^- \subseteq \hat{\mathcal{R}} \subseteq \mathcal{R}^0, \quad W(\mathcal{R}^-) \leq W(\hat{\mathcal{R}}) \leq W(\mathcal{R}^0). \quad (34)$$

The left inclusion follows from $R_i^+ \leq R_i + 2r$, while the right inclusion follows from $R_i \leq R_i^+$. Therefore, if one primary-optimal set over $\mathcal{R}^0$ already lies entirely in $\mathcal{R}^-$, calibration screening does not reduce the best achievable primary coverage. With $\tau_L = 1$, we use this upper-confidence gate as a finite-sample admissibility screen before budgeted selection.

C. Exact Primary Coverage under a Hard Budget

Once calibration fixes the eligible pool, the primary selection problem has a simple interpretation. Each detector has a deployment cost and covers a subset of source attack classes, while the hard budget may prevent us from selecting all detectors at once. We first characterize the difficulty of this problem and then show why the bitmask dynamic program used by SILOSTITCH still returns the exact primary optimum for the processed instances.

Theorem V.3 (Primary hardness and exact parameterized solver). Computing W^* is NP-hard, even when every candidate is eligible, every cost is one, and all class weights are uniform. Nevertheless, the bitmask dynamic program returns a nonempty feasible set S_0 such that

$$C_{\text{sem}}(S_0) = W^*. \quad (35)$$

It stores at most 2^C masks and examines at most $|\widehat{\mathcal{R}}|2^C$ state transitions. Thus, the primary stage is fixed-parameter tractable in the number of target classes C .

Proof. For hardness, take a Maximum Coverage instance with universe $\{1, \dots, C\}$, subsets $\Gamma_1, \dots, \Gamma_M$, and cardinality limit k . Set $b_i = 1$, $B = k$, and $w_c = 1/C$. Maximizing C_{sem} then solves that Maximum Coverage instance up to the constant factor $1/C$.

For exactness, process the eligible candidates one at a time. After the first t candidates, associate every reachable union mask h with the least-cost detector set that produces that mask. When we process the next candidate, every subset either excludes it and keeps an earlier state, or includes it and extends an earlier mask to $h \vee \Gamma_{t+1}$. If two sets produce the same mask, the cheaper one dominates the more expensive one because every future candidate changes the two masks in exactly the same way. By induction, the final table therefore contains the least-cost realization of every reachable mask. Scanning the masks whose costs do not exceed B returns exactly W^* .

A C -bit mask has at most 2^C values, and every eligible candidate extends each current mask at most once. If all feasible masks have zero weighted coverage, we return the least-cost feasible singleton so that the downstream stacker still receives a nonempty input without changing the primary value. $\square$

The transition count does not include witness reconstruction or the sorting used by the later refinement stage. In our implementation, we process at most 10^6 mask states. The exactness guarantee applies to every instance that falls within this implementation limit.

D. Secondary Objective and Local Refinement

The exact primary solver fixes the best source coverage that the budget can support, but many detector sets may still attain that same value. We use the secondary objective Φ to choose among them. Before stating the local-refinement guarantee, we first explain why we do not replace the ordered objective with one weighted sum.

For a fixed problem instance, define the set of achievable primary values as

$$\mathcal{V} = \{C_{\text{sem}}(S) : S \in \mathcal{F}_B\}. \quad (36)$$

When $\mathcal{V}$ contains at least two values, let Δ_{inst} be the smallest positive gap between them. Because $0 \leq \Phi(S) \leq 1$, any coefficient $K > 1/\Delta_{\text{inst}}$ makes every maximizer of $K C_{\text{sem}} + \Phi$ respect the lexicographic order for that one fixed instance. However, Δ_{inst} depends on the class weights and candidate pool and can become arbitrarily small. Therefore,

no universal finite K enforces the required priority across arbitrary deployments. The ordered formulation in Eq. 8 states the priority directly and avoids instance-specific scale tuning.

We next ask whether the secondary score has a useful diminishing-returns property: once a selected set already captures some behavior, adding another similar detector should help less. This standard set-function property is called submodularity.

Proposition V.4 (Secondary structure). For fixed calibration statistics, U , C_{pred} , and D_{rep} are normalized, monotone, submodular set functions. Consequently, their nonnegative weighted combination Φ has the same properties.

Proof. For U and C_{pred} , each summand applies the nondecreasing concave function h_τ to a nonnegative modular sum. Therefore, the marginal gain from adding the same candidate cannot increase as the selected set grows. For D_{rep} , target candidate k contributes $\max_{i \in S} \kappa_{i,k}$, whose marginal gain from adding candidate a is

$$\max \left\{ 0, \kappa_{a,k} - \max_{i \in S} \kappa_{i,k} \right\}.$$

As S grows, the existing maximum can only increase, so this gain can only decrease. Averaging and taking a nonnegative weighted sum preserve normalization, monotonicity, and submodularity. $\square$

This submodularity does not by itself give a global approximation guarantee for the deployed secondary refinement. The family of sets that preserve exact primary coverage,

$$\mathcal{F}^* = \{S \in \mathcal{F}_B : C_{\text{sem}}(S) = W^*\}, \quad (37)$$

does not have the simple structure required by standard submodular-knapsack results: removing a detector can destroy the required coverage, and an arbitrary exchange need not restore it. Therefore, familiar global guarantees such as $1-1/e$ do not directly apply after we require exact primary coverage.

For $S \in \mathcal{F}^*$, let $\mathcal{N}_1(S)$ contain every set obtained by removing one selected candidate and adding one unselected candidate while preserving budget feasibility and coverage W^* . We can then state what the deployed refinement does guarantee.

Theorem V.5 (Coverage preservation and one-swap certificate). Given an exact primary solution, the refinement procedure terminates after finitely many accepted operations and returns $\widehat{S} \in \mathcal{F}^*$ such that

$$\Phi(S') \leq \Phi(\widehat{S}) + \varepsilon_{\text{loc}}, \quad \forall S' \in \mathcal{N}_1(\widehat{S}). \quad (38)$$

Proof. The initial set has coverage W^* . Since W^* is already the maximum feasible coverage and C_{sem} is monotone, every feasible addition keeps that primary value. Every accepted replacement is also checked explicitly for budget feasibility and equality to W^* . Each accepted operation improves Φ by more than ε_{loc} . Because the candidate pool contains only finitely many subsets, the procedure must terminate. At termination,

the one-swap search has found no budget-feasible, coverage-preserving replacement whose gain exceeds the tolerance, which yields Eq. 38. $\square$

We deliberately state this as a local guarantee. The refinement examines additions and one-for-one replacements rather than all possible multi-detector exchanges. It therefore preserves the exact primary optimum and returns a set that is stable against the defined one-swap neighborhood, but it does not claim a globally optimal secondary score.

E. Effective Coverage under Client Mixture Shift

The source-coverage objective records which attack classes were represented in the historical source domains. It does not, by itself, guarantee that a detector supporting a class still performs well on that class after deployment. The held-out calibration records provide current evidence, but the client mixture observed during deployment may itself differ from the mixture seen during calibration. We therefore connect historical source support with current classwise utility and quantify how a change in client mixture can weaken that connection.

This analysis does not change which set SILOSTITCH selects. Instead, after selection, we use the already computed classwise calibration bounds to ask which historically covered classes also have conservative evidence of current competence. Let $\vartheta_c \in [0, 1]$ be a prespecified current-utility threshold for class c and define

$$C_{\text{cert}}(S; \boldsymbol{\vartheta}) = \sum_{c=1}^C w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1 \wedge a_{i,c} \geq \vartheta_c\}. \quad (39)$$

A class contributes to this value only when at least one selected detector both reports enough historical source support and has a conservative current classwise utility above the threshold.

For comparison, let $C_{\mu}^a(S; \boldsymbol{\vartheta})$ replace the condition $a_{i,c} \geq \vartheta_c$ by $\mu_{i,c}^a \geq \vartheta_c$, where $a \in \{\text{cal}, \text{dep}\}$ denotes calibration and deployment. Let $h = (j, c)$ index a client–class cell. We consider a mixture-shift model in which detector behavior inside each client–class cell remains stable while only the proportions of those cells change between calibration and deployment. Let $R_{i,h} \in [0, 1]$ be the risk of detector i in cell h , and let π_h^{cal} and π_h^{dep} denote the two mixtures. For $a \in \{\text{cal}, \text{dep}\}$, define

$$R_i^a = \sum_{j,c} \pi_{j,c}^a R_{i,j,c}, \quad \mu_{i,c}^a = \sum_j \pi_{j|c}^a (1 - R_{i,j,c}), \quad (40)$$

and let $\text{TV}(p, q) = \frac{1}{2} \|p - q\|_1$.

Theorem V.6 (Effective coverage under mixture shift). Suppose that the within-class client mixtures differ by at most η_c . On the simultaneous calibration event, every adaptively selected set $\hat{S}$ satisfies

$$\begin{aligned} C_{\mu}^{\text{cal}}(\hat{S}; \boldsymbol{\vartheta} + \boldsymbol{\eta} + 2\mathbf{r}) &\leq C_{\text{cert}}(\hat{S}; \boldsymbol{\vartheta} + \boldsymbol{\eta}) \\ &\leq C_{\mu}^{\text{dep}}(\hat{S}; \boldsymbol{\vartheta}) \leq C_{\text{sem}}(\hat{S}), \end{aligned} \quad (41)$$

where $(\mathbf{r})_c = r_c$ and $(\boldsymbol{\eta})_c = \eta_c$. Moreover, if $\text{TV}(\pi^{\text{dep}}, \pi^{\text{cal}}) \leq \eta$, then

$$R_i^{\text{dep}} \leq R_i^{\text{cal}} + \eta. \quad (42)$$

Proof. The simultaneous calibration result and the mixture-shift assumption give

$$\mu_{i,c}^{\text{cal}} \geq \vartheta_c + \eta_c + 2r_c \implies a_{i,c} \geq \vartheta_c + \eta_c \implies \mu_{i,c}^{\text{dep}} \geq \vartheta_c.$$

Combining these implications with the fixed historical-support condition $\Gamma_{i,c} = 1$, taking the existence condition over the selected detectors, and summing the class weights yields Eq. 41. For the final result, total variation bounds the change in the expectation of any function taking values in $[0, 1]$. Applying this fact to the client–class risks gives Eq. 42. $\square$

The theorem separates three quantities that serve different purposes. C_{sem} is the historical source-coverage value optimized during selection. C_{cert} keeps only the part of that coverage that also has conservative current calibration evidence. C_{μ}^{dep} describes the corresponding current utility after deployment mixture shift. Thus, SILOSTITCH guarantees the historical coverage objective exactly, while this separate post-selection analysis states when that historical coverage is also supported by current predictive evidence.

VI. EVALUATION

We evaluate whether selective reuse of frozen intrusion detectors can deliver the three properties targeted by our design: preserving source attack-class coverage, maintaining useful current detection quality, and reducing deployment communication. We also study when these benefits become weaker by varying source heterogeneity, secondary design choices, privacy strength, and problem scale. To keep the discussion organized, we answer five questions.

- **RQ1: Detection quality under a budget.** How much source coverage and current detection quality do we retain when only part of the full deployment cost is available?
- **RQ2: Client label heterogeneity.** How does stronger heterogeneity across source silos affect candidate construction, source coverage, and final detection quality?
- **RQ3: Selection signals and learners.** How sensitive are the selected sets to the secondary signals, source-coverage settings, and learning backends?
- **RQ4: Score randomization.** What predictive and security costs arise when we protect selected score blocks with stronger Laplace randomization?
- **RQ5: Scalability and runtime.** Does the exact primary solver remain correct and practical as the candidate pool and attack vocabulary grow, and what communication reduction do we obtain at the evaluated scale?

A. Experimental Setup

Datasets. We use four multiclass cybersecurity datasets: CIC DDoS2019 (DDoS) [26], CICMalDroid2020 (MalDroid) [27], CIC Darknet2020 (Darknet) [28], and CIRA CIC DoHBrw 2020 (DoH) [29]. For each task, we construct 66 source clients

and let each client contribute one frozen candidate detector. Keeping the candidate count fixed across datasets helps us separate dataset effects from changes in pool size.

We require every local candidate to be a multiclass detector, so each client receives at least two source classes and between 74 and 30,517 fitting records depending on the dataset and partition. Our main operating point uses a 50% selection-cost budget. If all candidate costs were identical, this budget would allow roughly 33 of the 66 detectors; in practice, the selected count depends on the logical byte cost b_i of each candidate. In RQ5, we separately vary the candidate pool from 16 to 256 to isolate solver scaling. Table II summarizes the four tasks.

Among the 13 classes observed in the reserved DDoS test set, one attack class is absent from all client-training partitions. We still include that class in macro F1 because it appears at test time, but we exclude it from the minimum evaluated class recall because no source detector could have learned it from the source data.

Protocol. For each dataset, we split the available training pool into disjoint 60%, 10%, and 30% partitions for source-detector fitting, selection calibration, and shared-stacker fitting, respectively. After we fit the source detectors, the 60% source partitions are no longer used by the integration procedure. This reproduces the paper’s intended setting: the frozen source models and their coarse metadata remain available, while the raw source records are no longer revisited.

We use the 10% calibration partition only to evaluate the frozen candidate pool and select $\hat{S}$. We use the 30% stacker-fitting partition only after $\hat{S}$ has been frozen. The test set remains sealed until both steps are complete. Unless stated otherwise, we repeat the complete protocol over five data splits, use LightGBM [30] for both local detectors and the shared stacker, set the selection-cost budget to 50%, use $n_{\min} = 2$, use inverse-square-root source-class weights, and assign equal weights to the three secondary components in Φ .

Baselines and reference point. We compare SILOSTITCH with two budget-matched selection rules and with the full detector pool. The first baseline is random selection. Within each data split, we generate 20 fixed random candidate orders and scan each order once, adding a candidate whenever its cost still fits the remaining budget. We average these 20 selections within the split. This baseline tells us whether an arbitrary budget-feasible subset is already sufficient.

The second baseline ranks candidates by mean calibrated Brier utility per selection-cost byte and greedily packs them in that order. This rule favors current average behavior but does not explicitly protect source class coverage. We use it to test the main reason for our ordered objective: a utility-driven selector may gain current predictive quality by discarding unique source coverage. Finally, we use all 66 detectors as the unconstrained reference corresponding to the 100% selection-cost point.

Metrics and statistics. We use macro F1 as the main detection metric because every class contributes equally. This is important for imbalanced security traffic, where a frequent class can otherwise hide weak performance on a rare attack class.

We also report accuracy, balanced accuracy, and minimum evaluated class recall. The last metric is the smallest recall among classes that both appear in the sealed test set and belong to the fixed source-training class set.

We report weighted source class coverage separately from these predictive metrics because it answers a different question: whether the selected set retains historical class support, not whether it predicts current records correctly. For the DDoS privacy experiment, we also report *attack leakage*, defined as the fraction of predictions labeled benign that are actually attacks. We compute communication using Eqs. 5–7. Unless stated otherwise, we report means and 95% Student t confidence intervals over five repeated splits. When two conditions share a split, we pair their observations, and we apply Holm correction within each comparison family fixed before analysis.

Experimental environment. We run the centralized replay on one Linux machine with Python 3.9.25, NumPy 1.26.4, scikit-learn 1.3.2, LightGBM 4.6.0, and XGBoost [31] 1.4.2. We record wall-clock runtime and the logical bytes exchanged by the evaluated protocol. The reported runtime therefore measures the centralized experimental replay, while the communication metric follows the logical client–coordinator workflow defined in the system model.

B. RQ1: Detection Quality under a Budget

We first ask whether the 50% cost constraint actually provides a useful deployment point. A good result should preserve the maximum source coverage allowed by the budget, reduce communication substantially, and keep the final shared detector close to the full-pool reference. We therefore compare the methods at the 50% operating point and then inspect the full budget sweep in Fig. 2.

Detection quality and source coverage. At the 50% operating point, SILOSTITCH consumes 49.5%–50.0% of the full-pool selection cost. Across all four tasks, its absolute mean macro-F1 gap from the full detector pool remains below 0.028. The gaps are at most 0.0052 on MalDroid, Darknet, and DoH, while DDoS has the largest gap at 0.0258. Thus, even the largest observed loss remains small relative to the roughly one-half reduction in deployment cost.

The budget sweep in Fig. 2 also shows that more selected cost does not automatically produce higher sealed-test macro F1. MalDroid improves most clearly as the budget increases, DoH changes only slightly, and DDoS and Darknet are non-monotonic. This is consistent with our design motivation: once source coverage is protected, adding more frozen detectors can introduce redundant or less useful current behavior rather than guaranteeing a better stacker.

We next check whether the communication savings hide a failure on weak classes. Relative to the full pool, the 50% selected set changes accuracy by at most 0.0046 and minimum evaluated class recall by at most 0.0061 on MalDroid, Darknet, and DoH. On DDoS, the accuracy gap is 0.0296 but the minimum evaluated class-recall gap is only 0.0094. Darknet is a useful cautionary case: even the full pool reaches only 0.0077

Table II

SCALE OF THE FOUR EVALUATION TASKS. EACH TASK USES 66 CLIENTS AND ONE FROZEN CANDIDATE DETECTOR PER CLIENT. EVALUATED CLASSES COUNT LABELS IN THE SEALED TEST SET; DDoS CONTAINS ONE TEST CLASS ABSENT FROM SOURCE TRAINING.

Dataset	Training samples	Test samples	Features	Evaluated classes
DDoS	3,355,966	1,127,526	79	13
MalDroid	8,118	3,480	470	5
Darknet	99,066	42,459	76	11
DoH	1,005,708	431,034	29	4

Table III

MACRO F1 AT THE 50% SELECTION-COST BUDGET. RANDOM SELECTION AND INDIVIDUAL-UTILITY RANKING RECEIVE THE SAME BUDGET; THE FULL DETECTOR POOL IS AN UNCONSTRAINED REFERENCE. ENTRIES REPORT MEANS AND 95% STUDENT t CONFIDENCE INTERVALS OVER FIVE REPEATED SPLITS.

Method	DDoS	MalDroid	Darknet	DoH
Full detector pool	0.6412 ± 0.0121	0.8449 ± 0.0123	0.5972 ± 0.0128	0.6073 ± 0.0042
Random selection	0.6079 ± 0.0287	0.8344 ± 0.0100	0.5931 ± 0.0071	0.6074 ± 0.0028
Individual utility ranking	0.6303 ± 0.0437	0.8374 ± 0.0132	0.5921 ± 0.0168	0.6069 ± 0.0042
SILOSTITCH	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038

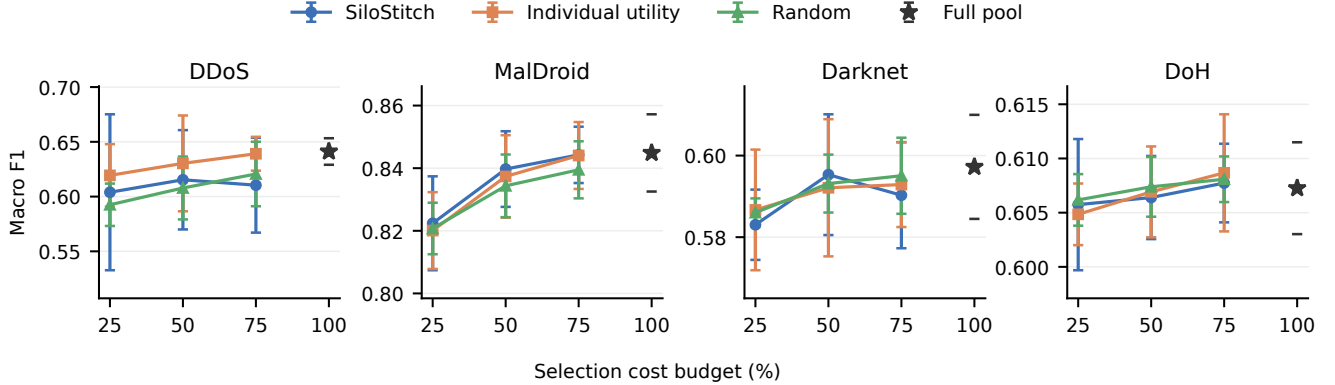


Figure 2. Macro F1 as a function of the selection-cost budget. We compare SILOSTITCH with the two budget-matched selection rules at 25%, 50%, and 75%; the star marks the unconstrained pool of 66 detectors. Error bars are 95% confidence intervals over five repeated splits after averaging the 20 random selections within each split. Each dataset uses its own vertical scale.

minimum evaluated class recall, compared with 0.0058 after selection. The poor weakest-class recall is therefore already present in the underlying candidate pool rather than being created by budgeted selection. Most importantly for our primary objective, SILOSTITCH reaches the maximum weighted source class coverage available under the 50% budget on all four datasets.

Why coverage priority matters. The comparison with individual-utility ranking shows why we do not optimize current average performance alone. On MalDroid and Darknet, SILOSTITCH has the higher mean macro F1, while the two methods are nearly identical on DoH. DDoS exposes the intended conflict most clearly: the utility-only baseline gains 0.0150 macro F1 over SILOSTITCH, but its weighted source coverage falls to 0.910, whereas SILOSTITCH retains the maximum value of 1.0. In other words, the utility-only selector buys a small current-score gain by dropping part of the historical attack-class coverage. Our ordered objective

explicitly rules out that exchange.

Overall, RQ1 shows the deployment advantage targeted by the paper: at roughly half the selection cost, we preserve the maximum source coverage available under the budget and keep macro-F1 loss below 0.028 across all four tasks. The DDoS comparison also provides a concrete case in which the source-coverage priority is active rather than merely formal.

C. RQ2: Client Label Heterogeneity

The previous experiment starts from a fixed source candidate pool. We now study how the heterogeneity that creates such a pool can affect the integration problem before and after selection. We separate two effects: whether we can construct valid multiclass source detectors at all, and, when we can, whether stronger source-label imbalance consistently reduces the final macro F1.

We evaluate both the original DDoS and Darknet client assignments and controlled partitions drawn from Dirichlet

distributions with $\alpha \in \{10, 1, 0.5, 0.1\}$, together with an IID allocation. Smaller α creates stronger label imbalance. For the controlled partitions, we require every accepted source client to contain at least two training classes so that its frozen candidate remains a multiclass detector. Figure 3 reports both partition feasibility and the resulting detection quality.

When heterogeneity limits candidate construction. Darknet remains feasible for all ten seeds at every evaluated heterogeneity level, even as the measured client-label divergence rises from 0.0008 under IID allocation to 0.3516 at $\alpha = 0.1$. MalDroid behaves differently because it has only five source classes spread across the same 66 clients. All ten IID and all ten $\alpha = 10$ partitions are usable, but only five $\alpha = 1$ partitions satisfy the two-class requirement. At $\alpha = 0.5$ and 0.1, none of the ten fixed seeds produces a valid partition despite allowing up to 100 generation attempts per seed. For these extreme settings, the bottleneck appears before SILOSTITCH runs: the source silos cannot produce the required multiclass candidate pool.

Quality after a valid pool exists. Conditional on a usable candidate pool, stronger heterogeneity does not produce a monotonic decline in macro F1. Darknet stays between 0.5895 and 0.5976 at the 50% budget, while MalDroid stays between 0.8328 and 0.8375 over the feasible range. These narrow descriptive ranges suggest that, in this controlled experiment, increasing source-label imbalance affects candidate-pool feasibility more clearly than it affects aggregate detection quality once a valid pool has already been constructed.

The original client layouts show a related effect at the class level. As the budget increases from 25% to 75%, Darknet macro F1 rises from 0.6162 to 0.6290, while its minimum evaluated class recall rises more substantially from 0.0452 to 0.0964. For DDoS, macro F1 changes from 0.6039 to 0.6104 over the same budget range, while minimum class recall increases from 0.0568 to 0.0773. Thus, additional deployment budget can be more visible in the weakest observed class than in an aggregate score.

When the hard coverage priority becomes inactive. We also remove only the explicit source-coverage priority while keeping the same secondary refinement. Across 30 comparisons on the original assignments and 225 comparisons on the controlled assignments, the two variants choose the same set and therefore obtain the same macro F1, minimum class recall, communication, and source coverage. This result does not mean that the coverage-first objective is unnecessary. Instead, it identifies the complementary case to DDoS in RQ1: when the secondary search already reaches the maximum available source coverage, the hard priority is non-binding; when a utility-oriented selector would trade coverage away, the priority changes the result and prevents that loss.

D. RQ3: Selection Signals and Learners

After the primary coverage value has been fixed, SILOSTITCH still needs to choose among coverage-equivalent detector sets and then fit the shared stacker. RQ3 therefore separates design choices at these two levels. We first remove the three

Table IV
MACRO F1 AT THE 50% SELECTION-COST BUDGET FOR EACH LOCAL-TO-SHARED LEARNER PAIR. ENTRIES ARE MEANS OVER FIVE SPLITS; BOLD MARKS THE LARGEST DESCRIPTIVE MEAN FOR EACH DATASET.

Dataset	XGB →XGB	LGBM →XGB	XGB →LGBM	LGBM →LGBM
DDoS	0.617	0.635	0.633	0.615
MalDroid	0.806	0.833	0.827	0.840
Darknet	0.578	0.585	0.592	0.595
DoH	0.592	0.591	0.607	0.606

secondary signals one at a time, then vary the source-coverage settings, and finally compare the local and shared learning backends.

Secondary selection signals. Recall that U measures record-level utility, C_{pred} measures conservative classwise utility, and D_{rep} measures how well the selected set represents the behavior of the eligible pool. In every ablation, we keep the exact primary source coverage fixed and remove only one of these secondary terms. At the 50% budget, removing U lowers DDoS macro F1 by 0.0190, while removing D_{rep} raises it by 0.0181. Across datasets and budgets, however, the direction of these changes is not consistent, and no single removal remains statistically significant after Holm correction. We therefore do not claim that any one secondary term provides a universal independent gain. Their role is narrower: together they rank candidate sets after the coverage requirement has already been satisfied.

Source-coverage settings. We next vary the minimum source count $n_{\text{min}} \in \{1, 2, 5, 10\}$ and replace the inverse-square-root class weights with uniform weights. Across two fixed Darknet layouts and two learner combinations, these changes leave the selected set, attained source coverage, macro F1, and minimum evaluated class recall unchanged. This robustness is useful because it shows that the reported Darknet result is not driven by one fragile choice of n_{min} or class-weighting rule. At the same time, the lack of change means these settings provide no empirical tuning benefit in the tested layouts, so we do not generalize that insensitivity beyond them.

Learning backends. Table IV shows that backend choice creates larger descriptive differences than the source-coverage settings above. On DDoS, LightGBM local detectors with an XGBoost shared learner give the largest mean. On MalDroid and Darknet, LightGBM gives the largest mean at both levels. On DoH, XGBoost local detectors with a LightGBM stacker lead by a small margin. After Holm correction, only the two matched DoH comparisons between shared learners remain statistically significant, and in both cases the LightGBM shared learner is better than its XGBoost counterpart. We therefore treat the learner backend as a deployment tuning choice rather than a source of SILOSTITCH’s core advantage.

Taken together, RQ3 supports the intended separation between the primary and secondary stages. The maximum source coverage is stable across these tested secondary variations, while the exact predictive ranking among coverage-equivalent

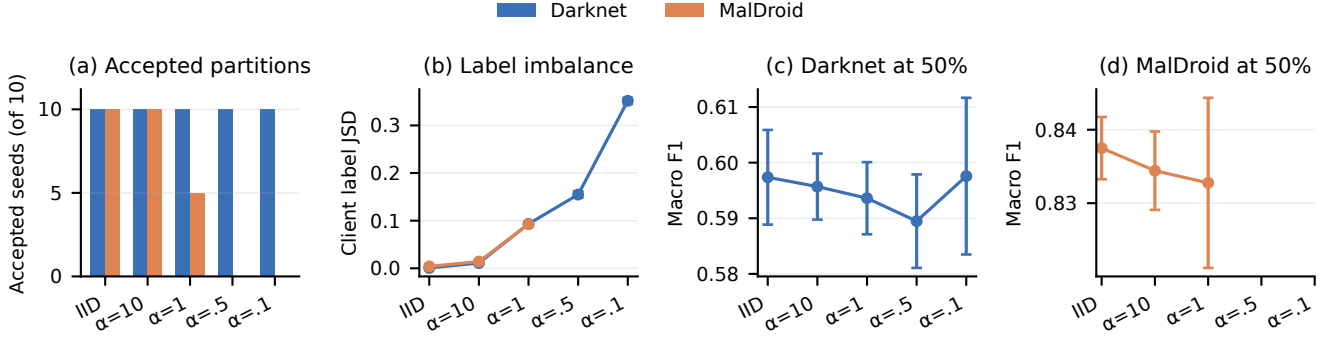


Figure 3. Effect of source-client label heterogeneity at the 50% selection-cost budget. Smaller Dirichlet α indicates stronger imbalance. Panel (a) counts valid partitions among ten fixed seeds, panel (b) reports client-label Jensen–Shannon divergence, and panels (c)–(d) report macro F1 for Darknet and MalDroid. Error bars are 95% confidence intervals over valid seeds; $n = 10$ except for MalDroid at $\alpha = 1$, where $n = 5$.

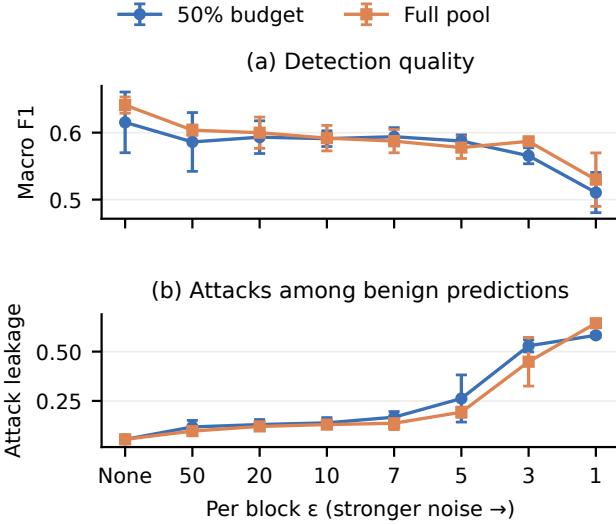


Figure 4. Effect of Laplace score randomization on DDoS. Panel (a) reports macro F1 and panel (b) reports attack leakage, the fraction of predictions labeled benign that are attacks. From left to right, the per-block privacy parameter decreases and noise becomes stronger. We compare SILOSTITCH at the 50% selection-cost budget with the full detector pool; error bars are 95% confidence intervals over the same five splits.

sets remains task dependent.

E. RQ4: Score Randomization

The previous experiments use unperturbed selected scores. We now enable the optional local privacy mechanism introduced in Eq. 24 and measure the utility cost of stronger randomization. We freeze the selected detector set and vary the per-block privacy parameter ϵ_b ; smaller ϵ_b means stronger Laplace noise. Theorem V.1 already gives the formal privacy guarantee, so here we focus on what the perturbation does to detection behavior.

Aggregate detection quality. Without added noise, the 50% selected set and the full detector pool obtain macro F1 values of 0.6153 and 0.6412, respectively. At the strongest evaluated

perturbation, $\epsilon_b = 1$, these values fall to 0.5105 and 0.5298. Relative to their own nonprivate settings, the selected set loses 0.1048 and the full pool loses 0.1114. The two configurations therefore suffer similar absolute degradation, which suggests that selecting a smaller detector set does not amplify the observed macro-F1 cost of score randomization.

Operational attack leakage. Macro F1 alone understates the security impact of the strongest perturbation. For the 50% selected set, attack leakage rises from 0.055 without noise to 0.262 at $\epsilon_b = 5$ and 0.583 at $\epsilon_b = 1$. For the full pool, it rises from 0.056 to 0.193 and then to 0.644. Thus, at $\epsilon_b = 1$, more than half of the predictions labeled benign are actually attacks under both configurations. This is operationally more severe than the aggregate macro-F1 curve alone suggests. We therefore report privacy together with both aggregate detection quality and attack leakage rather than treating one summary score as sufficient.

F. RQ5: Scalability and Runtime

Our primary solver is exact, but Theorem V.3 shows that its state space can grow exponentially with the number of target classes C . RQ5 therefore checks three practical questions: whether the implementation matches exhaustive search when enumeration is possible, how runtime changes as the candidate and class dimensions grow, and what end-to-end communication saving we obtain at the evaluated deployment scale.

Exactness audit. We first compare the dynamic program with exhaustive search at $C = 8$ and $M \in \{8, 10, 12, 16\}$. Across ten repetitions at each candidate-pool size, the solver reaches exactly the same primary coverage as exhaustive search in all 40 comparisons. Its median runtime ranges from 37.70 to 118.72 ms, and every 95th percentile remains below 125 ms. This experiment validates the implementation against an enumerable ground truth; it does not, by itself, claim that exhaustive search or the same runtime is practical at larger scales.

Scaling behavior. We next vary the two problem dimensions separately in Fig. 5. With 12 attack classes fixed, increasing

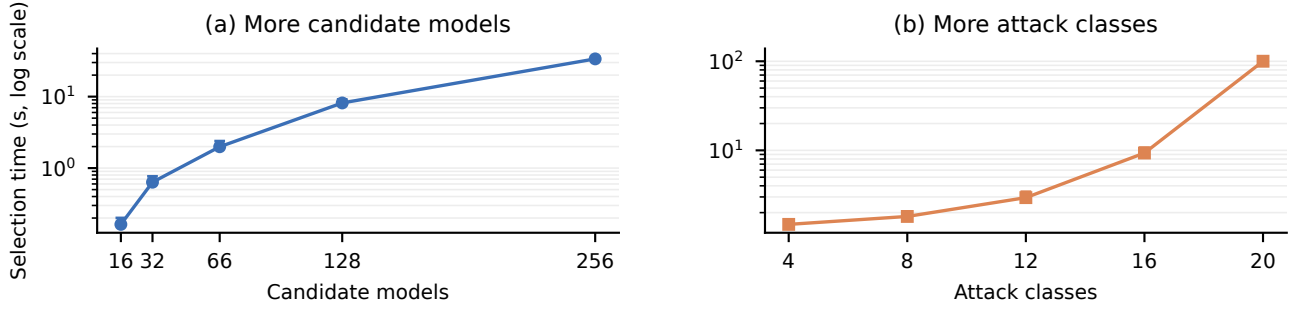


Figure 5. Scaling of the exact primary selector. Panel (a) fixes 12 attack classes and increases the candidate pool, while panel (b) fixes 66 candidates and increases the attack-class count. The vertical axis is logarithmic; points are medians and upper bars are 95th percentiles over ten repetitions.

Table V
SYSTEM OUTCOME AT THE 50% SELECTION-COST BUDGET.

Dataset	Full→selected (MiB)	Saved	$\Delta F1$
DDoS	7040.05→3489.07	50.4%	0.0258
MalDroid	662.66→335.94	49.3%	0.0052
Darknet	3436.78→1732.29	49.6%	0.0019
DoH	2942.76→1488.51	49.4%	0.0009

the candidate pool from 66 to 256 raises the median selection time from 1.99 to 33.73 s. With 66 candidates fixed, increasing the attack vocabulary from 12 to 20 classes raises the median from 2.95 to 100.10 s. The second increase is much sharper, which matches the structure of the bitmask solver: the candidate count enters linearly in the transition bound, while the class count controls the 2^C mask space. Thus, within our evaluated range, the attack vocabulary is the more visible scaling pressure near the upper end.

End-to-end deployment cost. Table V connects the solver study back to the main deployment objective. Across the four tasks, the 50% selection-cost setting reduces total logical communication by 49.3%–50.4% relative to the full pool, while the macro-F1 loss stays below 0.028. The three smallest gaps are at or below 0.0052; DDoS again has the largest gap but still retains the maximum weighted source class coverage from RQ1. Thus, the cost reduction reported in the abstract is not only a selector-level budget reduction: after we include model delivery, selected score upload, and the fixed protocol terms, the end-to-end logical communication is still approximately halved.

The total centralized replay time for one split, including every evaluated selection rule and budget, ranges from 0.78 to 50.09 minutes depending on the dataset. Stacker fitting dominates MalDroid and Darknet, while selection takes the largest share on DDoS. At the main scale of 66 candidates, the exact selector itself finishes in seconds. We therefore view the current practical limit as growth in the attack-class vocabulary rather than the 66-candidate deployment scale used in the main experiments.

VII. DISCUSSION AND CONCLUSION

In summary, we study selective integration for the setting in which raw historical source records are no longer available but frozen intrusion detectors and coarse source metadata remain authorized for reuse. With SILOSTITCH, we align heterogeneous outputs, maximize weighted source class coverage under a hard deployment budget, refine current predictive behavior without reducing that coverage, and fit one shared stacker from the selected outputs. Across four datasets, the 50% selection-cost setting reduces total logical communication by approximately half while keeping macro-F1 loss below 0.028 relative to the full detector pool.

When coverage priority matters. We use a strict coverage-first objective because source class coverage and current predictive quality answer different questions. The DDoS utility-only baseline shows a case in which this priority changes the result: it gains 0.0150 macro F1 but reduces weighted source coverage from 1.0 to 0.910. The heterogeneity ablation shows the complementary case in which the priority is non-binding because the secondary search already reaches maximum coverage. Together, these results clarify the role of the ordered objective: it does not force a different set when the current-behavior search already preserves all available coverage, but it prevents coverage loss when current utility would otherwise trade it away.

Our theoretical guarantees follow the same separation. The dynamic program solves the primary coverage problem exactly for every processed instance, while the subsequent refinement preserves that optimum and is only locally stable with respect to the defined one-swap neighborhood. We do not claim a globally optimal secondary score. Likewise, the simultaneous calibration theorem remains valid after adaptive detector selection for the observed calibration mixture. The separate mixture-shift result makes an additional assumption—stable client-class conditional behavior with bounded mixture change—when connecting historical source support to current deployment utility. A deployment outside those assumptions requires renewed calibration or validation rather than relying on the same post-selection certificate.

Deployment scope and scaling. The reported savings are logical bytes for the modeled workflow, including detector

delivery and aligned score uploads; they are not measurements of network latency or protocol overhead in a geographically distributed deployment. The centralized replay nevertheless lets us separate the computational stages under identical data splits. At 66 candidates, exact selection finishes in seconds, but runtime grows quickly with the attack-class count because the bitmask state space depends on 2^C . The current solver is therefore most suitable when the target attack vocabulary remains moderate; larger ontologies would require a different primary solver or an approximation that explicitly relaxes exactness.

Privacy scope and utility tradeoff. Our local privacy guarantee applies only to randomized selected score blocks. Public labels, contributed detector parameters, semantic maps, and source class-count metadata are outside that guarantee, and the absence of historical raw records should not itself be interpreted as a privacy guarantee. When enabled, the mechanism also introduces a measurable utility cost. The DDoS experiment shows why we examine this cost at more than one level: under strong perturbation, macro F1 degrades, but attack leakage reveals an even sharper operational failure in which more than half of benign-labeled predictions are attacks. A deployment should therefore tune score privacy using both aggregate predictive quality and security-critical error behavior.

Overall, our results show that when historical training records cannot be revisited, a fixed subset of existing intrusion detectors can still be integrated in a budget-aware way without treating historical source coverage and current predictive quality as interchangeable. SILOSTITCH turns that requirement into an explicit selection order and achieves substantial communication savings while making the remaining limits—current-data quality, privacy, and solver scale—visible rather than hiding them behind one aggregate objective.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2. MLSys, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html
- [3] T. Li, S. Hu, A. Beirami, and V. Smith, “Ditto: Fair and robust federated learning through personalization,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6357–6368. [Online]. Available: <https://proceedings.mlr.press/v139/li21h.html>
- [4] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, “Fedprompt: Communication-efficient and privacy-preserving prompt tuning in federated learning,” in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [5] C. S. Johnson, M. L. Badger, D. A. Waltermire, J. Snyder, and C. Skorupka, “Guide to cyber threat information sharing,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-150, 2016. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-150>
- [6] J. Xu, C. Li, Y. Zheng, and Z. Li, “ENTENTE: Cross-silo intrusion detection on network log graphs with federated learning,” in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2026-s93-paper.pdf>
- [7] E. GDPR, “General data protection regulation (gdpr),” *Cit. on*, vol. 4, 2018.
- [8] J. K.-W. Lee, P. Sattigeri, and G. W. Wornell, “Learning new tricks from old dogs: Multi-source transfer learning from pre-trained networks,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 4372–4382. [Online]. Available: <https://papers.nips.cc/paper/8688-learning-new-tricks-from-old-dogs-multi-source-transfer-learning-from-pre-trained-networks>
- [9] Y. Wei, Z. Hu, L. Shen, Z. Wang, Y. Li, C. Yuan, and D. Tao, “Task groupings regularization: Data-free meta-learning with heterogeneous pre-trained models,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 52 573–52 587. [Online]. Available: <https://proceedings.mlr.press/v235/wei24e.html>
- [10] Q. Dong, A. Muhammad, F. Zhou, C. Xie, T. Hu, Y. Yang, S.-H. Bae, and Z. Li, “ZooD: Exploiting model zoo for out-of-distribution generalization,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/cd305fdee96836d5cc1de94577d71b61-Abstract-Conference.html
- [11] X. Zhao, G. Sun, R. Cai, Y. Zhou, P. Li, P. Wang, B. Tan, Y. He, L. Chen, Y. Liang, B. Chen, B. Yuan, H. Wang, A. Li, Z. Wang, and T. Chen, “Model-GLUE: Democratized LLM scaling for a large model zoo in the wild,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://openreview.net/forum?id=M5JW7O9vc7>
- [12] Y. Birman, S. Hindi, G. Katz, and A. Shabtai, “Cost-effective ensemble models selection using deep reinforcement learning,” *Information Fusion*, vol. 77, pp. 133–148, 2022.
- [13] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, “CADE: Detecting and explaining concept drift samples for security applications,” in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2327–2344. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin>
- [14] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [15] Information Commissioner’s Office, “Guidance on AI and data protection,” Mar. 2023, version 2.0.17. [Online]. Available: <https://ico.org.uk/media/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/guidance-on-ai-and-data-protection-2-0.pdf>
- [16] S. Thirumuruganathan, M. Nabeel, E. Choo, I. Khalil, and T. Yu, “SIRAJ: A unified framework for aggregation of malicious entity detectors,” in *43rd IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 507–521.
- [17] T. Qi, F. Wu, C. Wu, L. He, Y. Huang, and X. Xie, “Differentially private knowledge transfer for federated learning,” *Nature Communications*, vol. 14, p. 3785, 2023. [Online]. Available: <https://www.nature.com/articles/s41467-023-38794-x>
- [18] W. Qiu, Y. Feng, Y. Li, Y. Chang, K. Qian, B. Hu, Y. Yamamoto, and B. W. Schuller, “Fed-MStacking: Heterogeneous federated learning with stacking misaligned labels for abnormal heart sound detection,” *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 9, pp. 5055–5066, Sep. 2024.
- [19] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, “Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 4002–4015, 2023.
- [20] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, “Low-quality training data only? a robust framework for detecting encrypted malicious network traffic,” in *Network and Distributed System Security Symposium*, 2024. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2024-81-paper.pdf>
- [21] R. Xie, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, and M. Zhang, “Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation,” in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim,

- CA: USENIX Association, 2023, pp. 625–642. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/xie>
- [22] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, “CoLD: Collaborative label denoising framework for network intrusion detection,” in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2026-s1950-paper.pdf>
- [23] S. Maddock, G. Cormode, T. Wang, C. Maple, and S. Jha, “Federated boosted decision trees with differential privacy,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2249–2263. [Online]. Available: <https://doi.org/10.1145/3548606.3560687>
- [24] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, “Local privacy and statistical minimax rates,” in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, 2013, pp. 429–438.
- [25] C. Dwork, F. McSherry, K. Nissim, and A. Smith, “Calibrating noise to sensitivity in private data analysis,” in *Theory of cryptography conference*. Springer, 2006, pp. 265–284.
- [26] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, “Developing realistic distributed denial of service (ddos) attack dataset and taxonomy,” in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.
- [27] S. Mahdavi, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, “Dynamic android malware category classification using semi-supervised deep learning,” in *IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 515–522.
- [28] A. Habibi Lashkari, G. Kaur, and A. Rahali, “Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning,” in *2020 the 10th International Conference on Communication and Network Security*, 2020, pp. 1–13.
- [29] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, “Detection of doh tunnels using time-series classification of encrypted traffic,” in *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 63–70.
- [30] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” *Advances in neural information processing systems*, vol. 30, pp. 3146–3154, 2017.
- [31] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.